Signal Processing in Computer Vision

ASSIGNMENT 1

January 7, 2026

Contents

1 The Mathematical Foundation

To manipulate digital images effectively, we must first understand the mathematics of signals. The core idea is that complex signals can be decomposed into simpler building blocks: sine and cosine waves.

1.1 Decomposing Signals: Fourier Series & Transform

The **Fourier Series** allows us to represent any *periodic* function as a weighted sum of sinusoids. However, most real-world signals (like an audio clip or an image) are not periodic; they happen once and then stop.

To handle these, we use the **Fourier Transform**. It stretches the period of the Fourier Series to infinity, allowing us to decompose *aperiodic* signals. This transformation takes us from the **Time Domain** (signal intensity over time) to the **Frequency Domain** (how much of each frequency is present).

$$F(\omega) = \int_{-\infty}^{\infty} f(t) e^{-j\omega t} dt \tag{1}$$

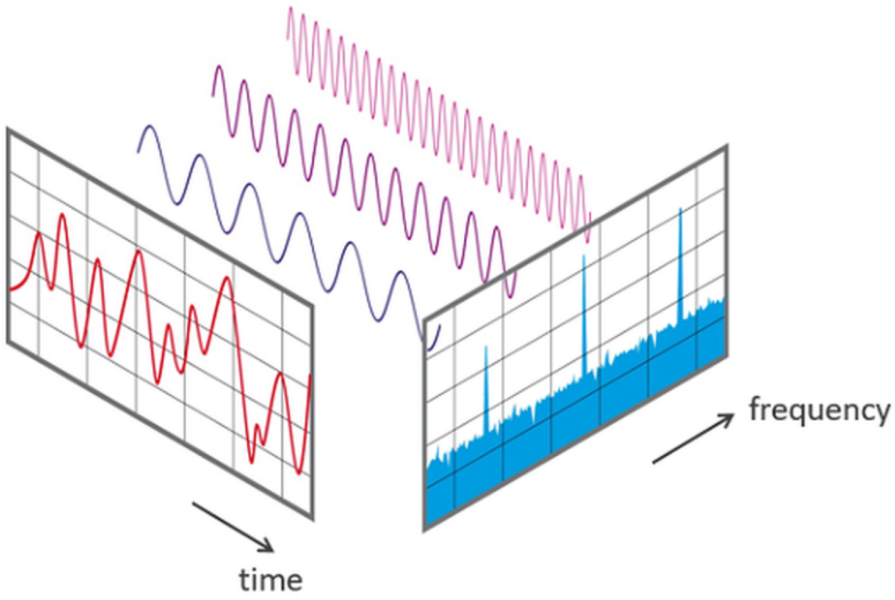

Figure 1: Visualizing Time Domain vs. Frequency Domain decomposition.

1.2 The Discrete World: DFT and FFT

Since computers cannot store continuous infinite waves, we must digitize signals.

- **DFT (Discrete Fourier Transform):** The mathematical operation used to calculate the frequency spectrum of a discrete set of samples (like pixels).
- **FFT (Fast Fourier Transform):** An algorithm effectively $O(N \log N)$ that computes the DFT. Without FFT, image processing would be too slow to be practical.

2 Digital Image Representation

2.1 Images as Matrices

A digital image is not a continuous canvas but a discrete matrix of numbers.

- **Grayscale:** A 2D matrix ($H \times W$). Values typically range from 0 (black) to 255 (white).
- **Color (RGB):** A 3D matrix ($H \times W \times 3$). The three layers correspond to Red, Green, and Blue channels.

Important Implementation Note: Libraries handle color channels differently. OpenCV loads images as **BGR**, while Matplotlib expects **RGB**. Always convert before displaying:

```
img_bgr = cv2.imread("image.jpg")
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
```

3 The Frequency Domain of Images

When we apply the FFT to a 2D image, we convert spatial data (pixels) into frequency data. But what does "frequency" mean in a static image?

- **Low Frequencies:** Large, smooth objects; background gradients; skin tones.
- **High Frequencies:** Sharp edges; fine details; noise; texture.

3.1 Magnitude and Phase

The output of an FFT is a matrix of complex numbers. This gives us two distinct pieces of information:

1. **Magnitude ($|F|$):** Tells us *how much* of a frequency is present. This dictates the "texture" or contrast of the image.
2. **Phase ($\angle F$):** Tells us *where* the waves align. This dictates the "structure" or geometry of the image.

4 Image Filtering and Convolution

4.1 The Convolution Theorem

This theorem is the bridge between the two domains. It states that **convolution in the spatial domain is equivalent to multiplication in the frequency domain**.

$$f(x,y) * h(x,y) \iff F(u,v) \cdot H(u,v)$$

This allows us to filter images efficiently: 1. Transform image to Frequency Domain. 2. Multiply by a filter mask (simple element-wise multiplication). 3. Transform back to Spatial Domain.

4.2 Constructing Filters (Masks)

To filter an image, we create a "Mask" matrix of the same size as the image.

4.2.1 Low Pass Filter (Blur)

We want to keep low frequencies (smooth parts) and remove high frequencies (edges).

- The low frequencies are in the center of the shifted FFT spectrum.
- **Mask:** A white circle (1s) in the center, surrounded by black (0s).

4.2.2 High Pass Filter (Edge Detection)

We want to remove the smooth background and keep only the sharp changes.

- **Mask:** A black circle (0s) in the center, surrounded by white (1s).
- Note: $Mask_{HPF} = 1 - Mask_{LPF}$

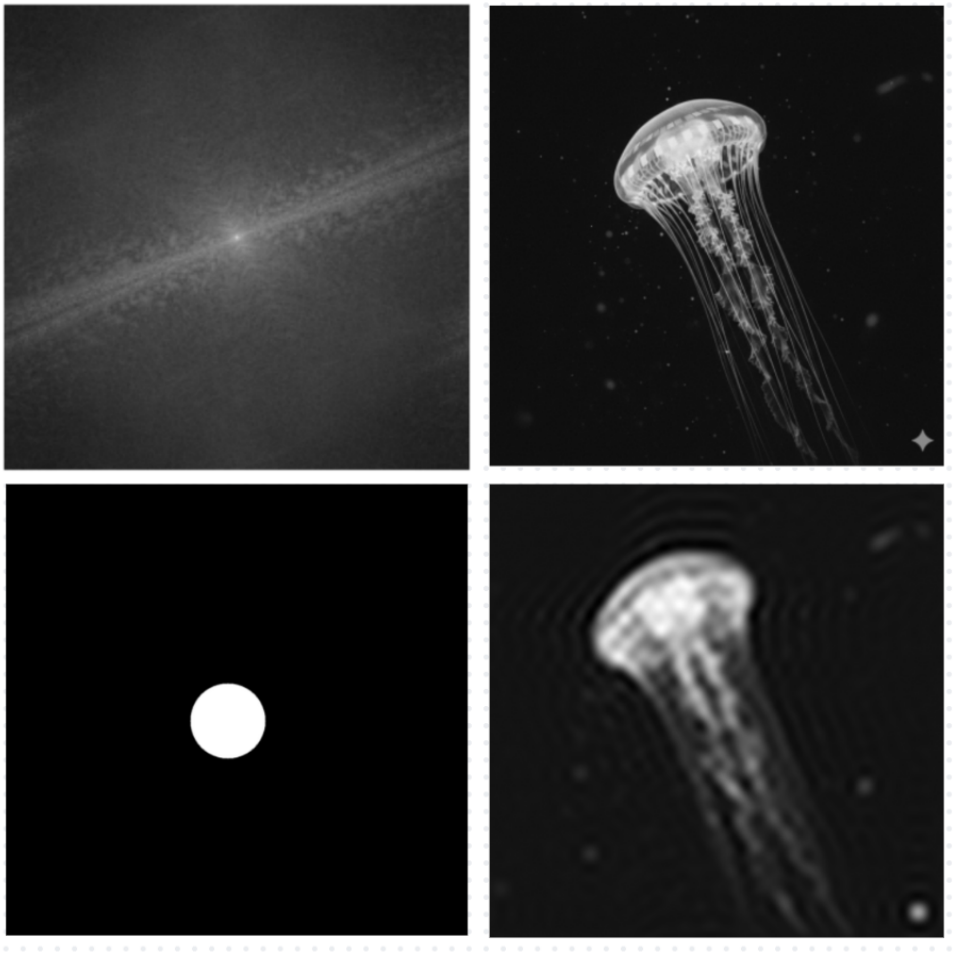

Figure 2: Applying a Low Pass Filter via FFT.

5 Implementation Pipeline

This section details the Python workflow required to implement the theory above, specifically for tasks like filtering or phase/magnitude swapping.

5.1 1. Pre-processing

Load the image and convert it to grayscale.

```
img = cv2.imread('input.png', 0) # 0 flag loads as grayscale
```

5.2 2. The Forward Transform

Compute the 2D FFT. We then shift the zero-frequency component (DC) from the top-left corner to the center of the image for easier masking.

```
f = np.fft.fft2(img)
fshift = np.fft.fftshift(f)
```

5.3 3. Masking (Filtering)

Create a mask and multiply it with the shifted spectrum.

```
rows, cols = img.shape
crow, ccol = rows//2, cols//2
mask = np.zeros((rows, cols), np.uint8)

# Creating a Low Pass Filter (Circle of 1s in center)
radius = 30
cv2.circle(mask, (ccol, crow), radius, 1, thickness=-1)

# Apply the mask
fshift_filtered = fshift * mask
```

5.4 4. The Inverse Transform

To view the result, we must reverse the process: (1) Inverse Shift, (2) Inverse FFT, (3) Magnitude Calculation.

```
f_ishift = np.fft.ifftshift(fshift_filtered)
img_back = np.fft.ifft2(f_ishift)
img_back = np.abs(img_back) # Complex to Real
```

Assignment 2: Image Processing using NumPy

January 7, 2026

Contents

1 Introduction

This assignment focuses on understanding core image processing techniques by implementing them using NumPy. The objective is to avoid high-level library shortcuts and instead develop a clear understanding of pixel-level operations such as histogram computation, color space conversion, creative filtering, and white balancing.

OpenCV is used strictly for image input/output and for validating results where required.

2 Part A: Image Histograms

2.1 Grayscale Histogram

2.1.1 Concept

A grayscale histogram represents the distribution of intensity values (0–255) in an image. It is useful for analyzing brightness, contrast, and exposure characteristics.

2.1.2 Methodology

- Initialize a histogram array with 256 bins
- Traverse the image pixel-by-pixel
- Increment the bin corresponding to each pixel intensity
- Visualize the histogram using Matplotlib

2.1.3 Illustrative Code Snippet

```
hist = np.zeros(256)
for i in range(height):
    for j in range(width):
        intensity = gray_img[i, j]
        hist[intensity] += 1
```

2.2 RGB Histogram

2.2.1 Concept

An RGB histogram consists of three separate histograms corresponding to the Red, Green, and Blue channels. Each histogram captures the intensity distribution of its respective channel.

2.2.2 Methodology

- Extract R, G, and B values for each pixel
- Maintain three independent 256-bin arrays
- Plot all three histograms together for comparison

2.2.3 Illustrative Code Snippet

```
r, g, b = img[i, j]
hist_r[r] += 1
hist_g[g] += 1
hist_b[b] += 1
```

3 Part B: RGB to HSV Conversion

3.1 Motivation

The HSV color space separates color information from brightness, making it more intuitive for color-based operations such as filtering, saturation control, and white balance.

3.2 Algorithm Overview

1. Normalize RGB values to the range [0,1]
2. Compute maximum, minimum, and difference (delta)
3. Calculate Hue based on the dominant color channel
4. Scale HSV values to match OpenCV conventions

3.3 Hue Computation Logic

The hue value depends on which RGB channel has the maximum intensity.

```
if cmax == r:
    h = 60 * ((g - b) / delta)
elif cmax == g:
    h = 60 * ((b - r) / delta + 2)
else:
    h = 60 * ((r - g) / delta + 4)
```

3.4 OpenCV Compatibility

To ensure compatibility with `cv2.cvtColor`, the HSV values are scaled as follows:

- Hue: 0–179
- Saturation: 0–255
- Value: 0–255

Correctness is verified by converting the generated HSV image back to RGB using OpenCV.

4 Part C: Creative Filters

Two creative filters were designed, each incorporating multiple color modifications to produce visually distinct and aesthetically pleasing results.

4.1 Filter 1: *Sunburn Deluxe*

4.1.1 Description

This filter enhances warmth by increasing brightness and boosting red tones, producing a vibrant and warm look.

4.1.2 Suitable Images

Sunsets, beach scenes, and outdoor portraits.

4.1.3 Illustrative Code Snippet

```
img = img + 20          # brightness increase
img[:, :, 0] *= 1.1     # red channel boost
```

4.2 Filter 2: *Sad Monsoon*

4.2.1 Description

This filter creates a cool and muted appearance by reducing saturation and adding a blue tint.

4.2.2 Suitable Images

Rainy scenes, foggy landscapes, and overcast environments.

4.2.3 Illustrative Code Snippet

```
img = img * 0.8          # saturation reduction
img[:, :, 2] += 15      # blue tint
```

4.3 Design Rationale

Combining multiple subtle adjustments produces more natural results than applying a single aggressive modification.

5 Part D: White Balance

5.1 White Patch Method

5.1.1 Concept

The white patch method assumes that the brightest pixel in the image should appear white. The image is scaled accordingly to correct overall color bias.

5.1.2 Illustrative Code Snippet

```
max_val = np.max(img)
scale = 255 / max_val
balanced = img * scale
```

5.2 Adjustable White Balance

5.2.1 Concept

A user-controlled parameter allows interpolation between colder and warmer versions of the image, with the original image retained at the midpoint.

5.2.2 Illustrative Code Snippet

```
output = (1 - value) * cold_img + value * warm_img
```

5.2.3 Design Insight

Blue tones are associated with colder lighting, while red and yellow tones convey warmth, making this interpolation perceptually meaningful.

6 Conclusion

This assignment emphasizes understanding image processing fundamentals through low-level NumPy-based implementations. The experiments demonstrate how simple pixel manipulations can significantly influence visual perception.

Contents

1 Introduction

1.1 Aim of the Assignment

The aim of this assignment is to implement fundamental classical image processing operations from scratch and understand how images are processed at the pixel level. The assignment focuses on manually implementing convolution-based techniques such as blurring, edge detection, sharpening, frequency analysis, and colour-based processing using NumPy. Another objective is to study how different kernels affect image features in both spatial and frequency domains, and to verify these implementations by comparing them with standard OpenCV functions where allowed.

2 Part A: Convolution and Blurring

2.1 Convolution in Image Processing

2.1.1 Definition and Kernel Concept

Convolution is a mathematical operation in which an image is processed using a small matrix called a kernel or filter. The kernel is placed over a local region of the image, corresponding pixel values are multiplied, and the results are summed to produce a new pixel value. Different kernels are used to achieve effects such as blurring, edge detection, and sharpening.

2.1.2 Sliding Window and Kernel Effects

The kernel moves across the image in a sliding window manner, processing one local region at a time. The output of convolution depends on the kernel values: kernels with positive values smooth the image and reduce noise, while kernels containing both positive and negative values highlight intensity changes and edges.

2.2 Padding in Convolution

2.2.1 Padding and Boundary Handling

When convolution is applied near image boundaries, neighbouring pixels may fall outside the image. Padding is used to extend the image so that convolution can be computed at all pixel locations while preserving the original image size. In this assignment, reflect padding was used, where boundary pixel values are mirrored. This approach avoids artificial edge artifacts and produces smoother and more consistent results near image borders.

2.3 Grayscale Conversion

2.3.1 Need for Grayscale Images

Most image processing operations operate on intensity values rather than colour information. Converting an RGB image to grayscale reduces the image to a single channel, simplifying computation while retaining important structural features such as edges and textures.

2.4 Convolution Implementation (NumPy Only)

2.4.1 Implementation Overview

Convolution was implemented manually using a custom NumPy-based function without relying on built-in filtering utilities. For a kernel of size $k \times k$, reflect padding of size $\lfloor k/2 \rfloor$ was applied to handle boundary pixels. At each pixel location, a local image region was extracted, multiplied element-wise with the kernel, and summed to compute the output value. This process was repeated across the image using a sliding window approach.

2.4.2 Implementation of Convolution Code

The complete NumPy implementation of the convolution function is shown below.

```python
import numpy as np

def convolve(img, kernel):
    k = kernel.shape[0]
    pad = k // 2

    # pad image
    padded = np.pad(img, pad_width=pad, mode='reflect')

    h, w = img.shape
    output = np.zeros((h, w), dtype=np.float32)

    # slide kernel
    for i in range(h):
        for j in range(w):
            region = padded[i:i+k, j:j+k]
            output[i, j] = np.sum(region * kernel)

    return output
```

Figure 1: Custom NumPy implementation of the convolution function

2.5 Average Blur (Mean Filtering)

2.5.1 Box Kernel

Average blur is implemented using a box kernel where all elements have equal values. For a kernel of size $k \times k$, each element has a value of $1/k^2$. This ensures that all neighbouring pixels contribute equally to the output pixel.

2.5.2 Effect on Noise and Details

Average blur smooths the image by reducing random noise and small intensity variations. However, since all pixels are treated equally, fine details and sharp edges are also blurred. As a result, average blur is effective for noise reduction but performs poorly at preserving edges.

2.5.3 Average Blur on Grayscale Image

When applied to a grayscale image, average blur smooths intensity variations across the image. Noise is reduced, but edges become less sharp due to uniform averaging. This effect becomes stronger as the kernel size increases.

```
def average_blur_gray(img, ksize):
    kernel = np.ones((ksize, ksize)) / (ksize * ksize)
    return convolve(img, kernel)
```

Figure 2: NumPy implementation of average blur for grayscale images

2.5.4 Average Blur on RGB Image

For RGB images, average blur is applied separately to each colour channel. The blurred channels are then combined to form the final output image. This approach preserves colour consistency while smoothing spatial variations in all channels.

```
def average_blur_rgb(img, ksize):
    output = np.zeros_like(img, dtype=np.float32)

    for c in range(3):  # R, G, B
        output[:, :, c] = average_blur_gray(img[:, :, c], ksize)

    return output
```

Figure 3: NumPy implementation of average blur applied channel-wise on RGB images

2.6 Gaussian Blur

2.6.1 Gaussian Distribution

Gaussian blur is based on the Gaussian (normal) distribution, which assigns higher weights to pixels closer to the center and lower weights to pixels farther away. This results in smoother blurring while preserving edges better than simple average blur.

The two-dimensional Gaussian function is defined as:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

Here, σ represents the standard deviation of the distribution and controls the amount of blurring. A larger value of σ produces stronger smoothing.

2.6.2 Gaussian Kernel Generation

A Gaussian kernel is generated by evaluating the Gaussian function at each location in a $k \times k$ window centered at zero. Each kernel element represents the weight assigned to the corresponding neighboring pixel during convolution.

2.6.3 Kernel Normalization

After computing the Gaussian kernel values, normalization is performed by dividing each element by the sum of all kernel values. This ensures that the overall intensity of the image remains unchanged after convolution.

2.6.4 Implementation: Gaussian Kernel

The Gaussian kernel was implemented using NumPy by explicitly computing the Gaussian function for each kernel position and normalizing the kernel.

```python
def gaussian_kernel(ksize, sigma):
    k = ksize // 2
    kernel = np.zeros((ksize, ksize), dtype=np.float32)

    for i in range(-k, k+1):
        for j in range(-k, k+1):
            kernel[i+k, j+k] = np.exp(-(i*i + j*j) / (2 * sigma * sigma))

    # normalize
    kernel /= np.sum(kernel)
    return kernel
```

Figure 4: NumPy implementation of Gaussian kernel generation

2.6.5 Gaussian Blur on Grayscale Image

Gaussian blur on a grayscale image is obtained by convolving the image with the generated Gaussian kernel using the custom convolution function. This operation reduces noise while maintaining smoother transitions near edges.

```python
def gaussian_blur_gray(img, ksize, sigma):
    kernel = gaussian_kernel(ksize, sigma)
    return convolve(img, kernel)
```

Figure 5: Gaussian blur implementation on grayscale image

2.6.6 Gaussian Blur on RGB Image

For RGB images, Gaussian blur is applied independently to each color channel (Red, Green, and Blue). The blurred channels are then combined to obtain the final RGB output image.

```python
def gaussian_blur_rgb(img, ksize, sigma):
    output = np.zeros_like(img, dtype=np.float32)

    for c in range(3):
        output[:, :, c] = gaussian_blur_gray(img[:, :, c], ksize, sigma)

    return output
```

Figure 6: Gaussian blur applied channel-wise on RGB image

2.7 Comparison: Average Blur vs Gaussian Blur

2.7.1 Kernel Structure

The average blur uses a box kernel where all neighboring pixels are given equal weight. In contrast, the Gaussian blur uses a weighted kernel where pixels closer to the center have higher importance. This difference in kernel structure leads to different smoothing behavior in the output images.

2.7.2 Edge Preservation

Average blur reduces noise by uniformly averaging pixel values, but it often removes fine details and edges. Gaussian blur provides smoother results by reducing noise while preserving important edge information due to its weighted nature.

2.7.3 Visual Comparison

A visual comparison was performed by applying both average blur and Gaussian blur to the same input image. The results clearly demonstrate that Gaussian blur preserves edges better while still achieving effective smoothing.

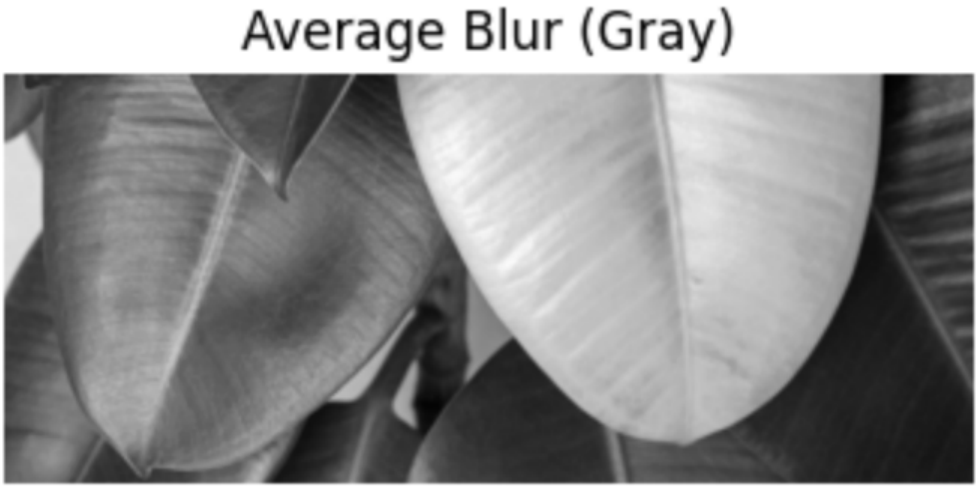

Figure 7: Result of Average Blur applied on the image

Figure 8: Result of Gaussian Blur applied on the image

3 Part B: Edge Detection

3.1 Sobel Edge Detection (NumPy Only)

3.1.1 Sobel Kernels

The Sobel X kernel detects intensity changes along the horizontal direction and highlights vertical edges in the image. The Sobel Y kernel detects intensity changes along the vertical direction and highlights horizontal edges in the image.

3.1.2 Computation of Gx and Gy

The grayscale image is convolved separately with the Sobel X and Sobel Y kernels to obtain the horizontal and vertical gradient components.

3.1.3 Gradient Magnitude Calculation

The overall edge strength is computed by combining the horizontal and vertical gradients using their magnitude. This produces a single edge intensity map. The gradient magnitude is normalized to the range 0–255 so that it can be displayed as a standard grayscale image.

3.1.4 Thresholding to Suppress Weak Edges

A threshold is applied to remove weak edges caused by noise, retaining only the most prominent edge structures.

3.2 Sobel Edge Detection Results

3.2.1 Sobel Gx Output

The Sobel X operator highlights vertical edges by detecting intensity changes along the horizontal direction.

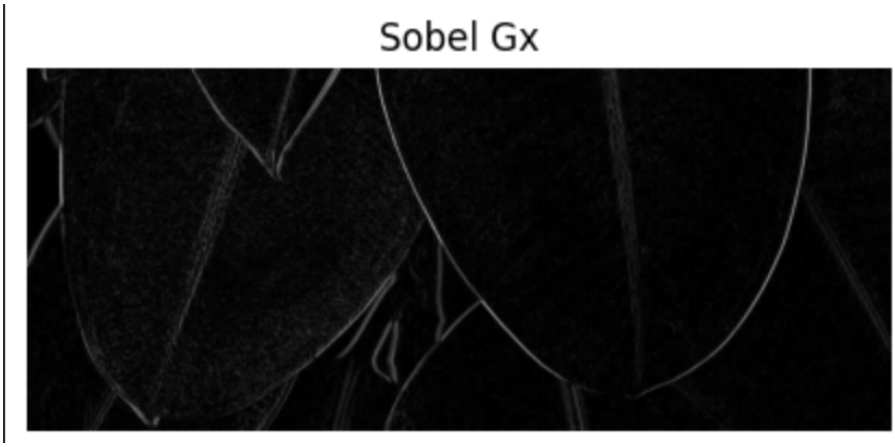

Figure 9: Sobel X (Gx) output

3.2.2 Sobel Gy Output

The Sobel Y operator highlights horizontal edges by detecting intensity changes along the vertical direction.

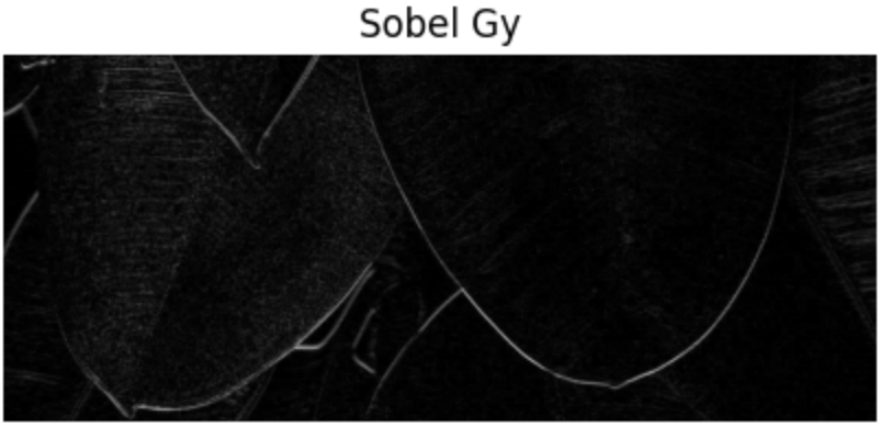

Figure 10: Sobel Y (Gy) output

3.2.3 Final Edge Map

The final edge map is obtained by combining the Gx and Gy outputs and applying thresholding.

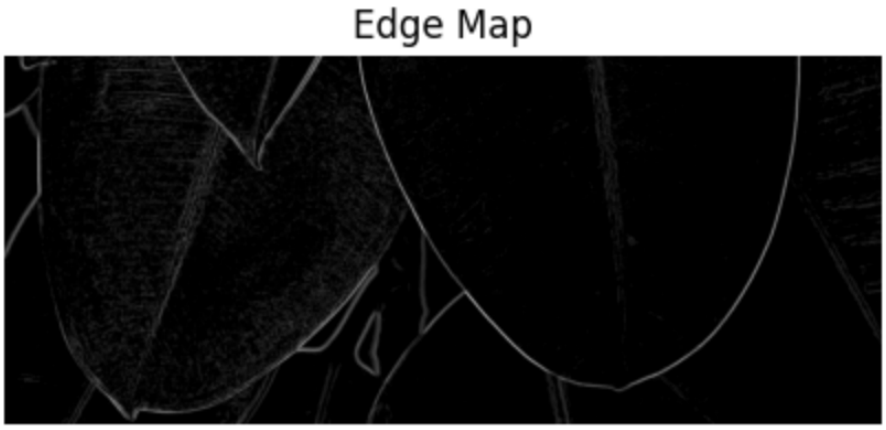

Figure 11: Final Sobel edge detection result

4 Part C: Image Sharpening

4.1 Laplacian Operator

4.1.1 First-order vs Second-order Derivatives

First-order derivatives detect edges by measuring intensity changes. Second-order derivatives, such as the Laplacian, detect rapid changes in gradients and highlight edges more sharply.

4.1.2 Laplacian Kernel (4-connected / 8-connected)

The Laplacian operator is implemented using a kernel that considers either 4-connected or 8-connected neighboring pixels. These kernels emphasize regions where intensity changes sharply in all directions.

4.1.3 Properties of Laplacian

The Laplacian produces near-zero values in smooth regions and strong positive or negative values near edges. This makes it effective for enhancing fine details and sharp transitions in an image.

4.2 Laplacian Sharpening (NumPy Only)

4.2.1 Laplacian Computation

The Laplacian of the image is computed by convolving the grayscale image with a Laplacian kernel. This operation highlights regions with rapid intensity changes.

4.2.2 Sharpening Formula

Image sharpening is performed by subtracting the Laplacian from the original image. This enhances edges and fine details while retaining the overall structure of the image.

4.2.3 Effect of Alpha Parameter

A scaling factor α is used to control the strength of sharpening. Larger values of α increase edge enhancement, while smaller values produce milder sharpening.

4.2.4 Output for Different Alpha Values

The sharpening effect was observed for different values of α to analyze its impact on image clarity.

4.3 Verification with OpenCV

4.3.1 Laplacian Sharpening Using `cv2`

To verify the NumPy-based implementation, Laplacian sharpening was also performed using OpenCV functions. This provides a reference output for comparison.

Figure 12: Laplacian sharpening result using OpenCV

4.3.2 Comparison with NumPy Result

The output obtained using the custom NumPy implementation closely matches the OpenCV result. This confirms the correctness of the manual convolution and sharpening approach.

Figure 13: Laplacian sharpening result using NumPy implementation

4.3.3 Matrix-level Verification

Both implementations produce similar edge enhancement patterns and intensity transitions. Minor differences arise due to internal normalization and data type handling.

4.4 Unsharp Masking

Unsharp masking is a sharpening technique that enhances edges by adding back high-frequency details to the original image. First, a blurred version of the image is created. The difference between the original image and the blurred image is called the mask. This mask is then added back to the original image to produce a sharpened result.

Figure 14: Result of unsharp masking

5 Part D: Frequency Domain Analysis

5.1 Fourier Transform in Image Processing

The Fourier Transform converts an image from the spatial domain to the frequency domain. In the frequency domain, low frequencies represent smooth regions, while high

frequencies represent edges and fine details. This representation helps in understanding how blurring and sharpening affect different frequency components of an image.

5.2 Magnitude Spectrum Analysis

The magnitude spectrum is obtained by computing the Fast Fourier Transform (FFT) of the image and shifting the zero frequency component to the center. A logarithmic scale is used to visualize the frequency components clearly.

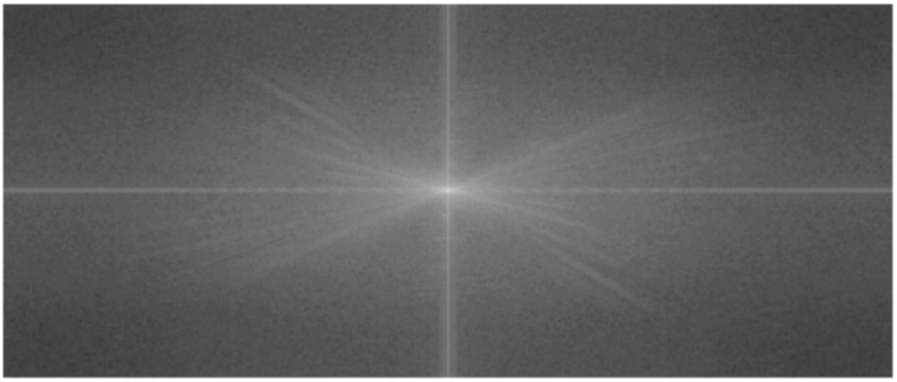

Figure 15: Magnitude spectrum of the original image

5.3 Low-Pass and High-Pass Filtering Verification

Gaussian blur acts as a low-pass filter by suppressing high-frequency components. Unsharp masking acts as a high-pass operation by amplifying high-frequency details. This behavior can be verified by comparing the magnitude spectra of the filtered images.

Figure 16: Magnitude spectrum after Gaussian blur

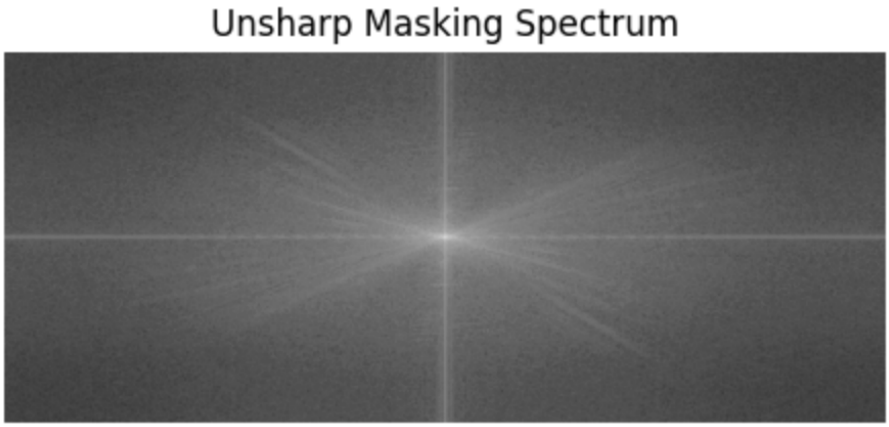

Figure 17: Magnitude spectrum after unsharp masking

6 Part E: Custom Colour Kernel

6.1 Colour-Based Edge Detection

Instead of relying only on intensity changes, colour-based edge detection uses differences between colour channels. In this task, colour information was used to highlight edges of specific regions in the image, such as the yellow leaf.

6.2 Custom Colour Kernel Design

A custom kernel was designed by exploiting the difference between colour channels. The response was strengthened by subtracting the green channel from the red channel, which highlights yellow regions effectively. This approach allows selective edge detection based on colour characteristics.

6.3 Detection of Yellow Leaf Edges

The responses from the custom colour kernel were combined to generate an edge map of the yellow leaf. Post-processing was applied to remove weak responses and retain clear boundaries.

Figure 18: Detected edges of the yellow leaf using custom colour kernel

7 Bonus Part F: Colour Manipulation

7.1 RGB to HSV Colour Space

The RGB colour space is not ideal for colour-based segmentation. HSV separates colour information (Hue) from intensity, making it easier to isolate specific colours. The RGB image was converted to HSV to simplify detection of the yellow leaf.

7.2 Yellow Leaf Detection Using HSV

A mask was created by applying thresholding on the Hue channel to isolate yellow pixels. This mask identifies only the yellow leaf region while ignoring the background.

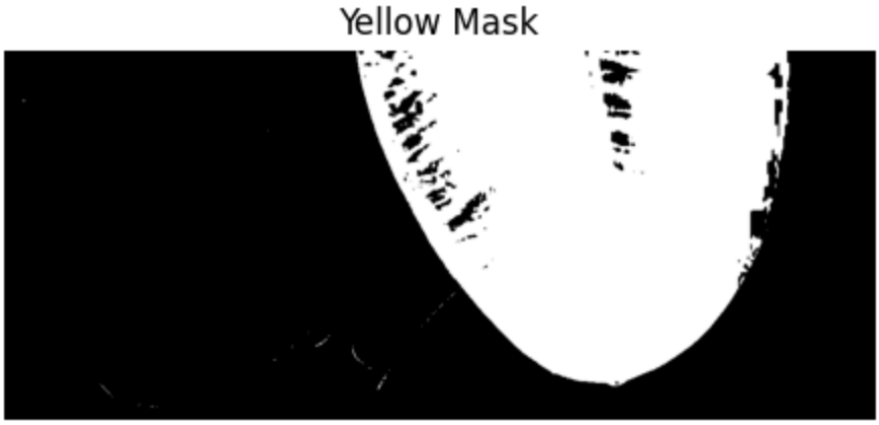

Figure 19: HSV mask highlighting the yellow leaf

7.3 Colour Manipulation

Using the HSV mask, the colour of the yellow leaf was changed to red by modifying the Hue values. The modified HSV image was then converted back to RGB to obtain the final result.

Figure 20: Final image after changing leaf colour from yellow to red

8 Observations and Discussion

It was observed that convolution-based operations strongly depend on the choice of kernel. Average blur effectively reduces noise but significantly blurs edges, whereas Gaussian blur provides smoother results with better edge preservation. Edge detection using Sobel operators successfully highlighted intensity transitions in the image. Laplacian sharpening and unsharp masking enhanced fine details by amplifying high-frequency components. Frequency domain analysis confirmed that blurring suppresses high frequencies while sharpening amplifies them. Colour-based processing using RGB and HSV spaces allowed selective detection and modification of specific regions in the image.

9 Conclusion

This assignment provided hands-on experience with classical image processing techniques implemented from scratch using NumPy. Through convolution, blurring, edge detection, sharpening, frequency analysis, and colour manipulation, a clear understanding of pixel-level image transformations was developed.

Assignment 4: Computer Vision - Corner Detection, Feature Detection, and Line Detection

1 Introduction

This report covers Assignment 4 in Computer Vision, focusing on three fundamental image processing techniques: corner detection using gradient analysis, ellipse fitting for feature characterization, and Hough transforms for line detection. These techniques form the backbone of feature extraction and object detection in computer vision applications.

2 Part A: Corner Detection Using Image Gradients

2.1 Theoretical Background

Corner detection is a crucial step in feature extraction. A corner is defined as a point where the gradient direction changes significantly in multiple directions. To detect corners, we need to compute image gradients in both horizontal and vertical directions.

2.2 Gradient Computation

We use the Sobel operators to compute the image gradients. The Sobel kernels are:

$$G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \quad G_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

The horizontal gradient I_x and vertical gradient I_y are computed by convolving the image with these kernels:

$$I_x = G_x \otimes I, \quad I_y = G_y \otimes I$$

where $\otimes$ denotes convolution and I is the grayscale image.

2.3 Gradient Visualization

The gradients I_x and I_y are then visualized separately to understand the edge structure in the image. For images with:

- **Flat regions**: Both I_x and I_y are close to zero (no edges)
- **Edge regions**: One gradient is significant while the other is minimal
- **Corner regions**: Both gradients are significant, indicating direction change

2.4 Scatter Plot Analysis

A scatter plot of I_y vs I_x reveals the relationship between horizontal and vertical gradients:

$$\text{Scatter}(I_x, I_y) = \{(I_x(i,j), I_y(i,j)) \mid \forall (i,j) \in \text{Image}\}$$

For corner detection, we observe points that are far from both axes, indicating significant changes in both directions. Thresholding the scatter plot helps identify potential corner candidates:

$$\text{Corners} = \{(i,j) \mid |I_x(i,j)| > \tau \text{ and } |I_y(i,j)| > \tau\}$$

where τ is an adaptive threshold determined from the gradient magnitude distribution.

2.5 Implementation Details

The implementation follows these steps:

```
import cv2
import numpy as np

def compute_gradients(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Compute Sobel derivatives
    Ix = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
    Iy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

    return Ix, Iy

def visualize_gradients(Ix, Iy):
    # Normalize for visualization
    Ix_norm = cv2.normalize(Ix, None, 0, 255, cv2.NORM_MINMAX).
       astype(np.uint8)
    Iy_norm = cv2.normalize(Iy, None, 0, 255, cv2.NORM_MINMAX).
       astype(np.uint8)

    return Ix_norm, Iy_norm
```

3 Part B: Harris Corner Detection and R-Map Generation

3.1 Harris Corner Detection Theory

Harris corner detection extends gradient analysis by creating a response map based on local changes. The method uses the structure tensor (also called the Harris matrix):

$$M = \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}$$

Smoothing these elements with a Gaussian window produces the Harris matrix:

$$M_{\text{smooth}} = \begin{pmatrix} S_{xx} & S_{xy} \\ S_{xy} & S_{yy} \end{pmatrix}$$

where:

$$S_{xx} = G(x,y) * I_x^2, \quad S_{yy} = G(x,y) * I_y^2, \quad S_{xy} = G(x,y) * I_x I_y$$

and $G(x,y)$ is a Gaussian window (typically 3×3 or 5×5).

3.2 Eigenvalue Analysis

The eigenvalues λ_1 and λ_2 of the Harris matrix characterize the local structure:

$$\lambda_1, \lambda_2 = \frac{\text{trace}(M) \pm \sqrt{\text{trace}(M)^2 - 4 \cdot \det(M)}}{2}$$

where:

$$\text{trace}(M) = S_{xx} + S_{yy}, \quad \det(M) = S_{xx}S_{yy} - S_{xy}^2$$

3.3 Harris Response Function

The Harris corner response is computed without explicitly calculating eigenvalues:

$$R = \det(M) - k \cdot \text{trace}(M)^2$$

where k is the Harris detector parameter (typically $k = 0.04$). The response function R has specific interpretations:

- $\lambda_1 \gg \lambda_2$ or $\lambda_2 \gg \lambda_1$: **Edge** ($R < 0$)
- λ_1 and λ_2 both large: **Corner** ($R > 0$, large value)
- λ_1 and λ_2 both small: **Flat** ($R \approx 0$)

3.4 R-Map Generation Algorithm

The R-map is generated using a sliding window approach:

```
def compute_harris_response(Ix, Iy, k=0.04):
    # Compute second moments
    Ixx = Ix**2
    Iyy = Iy**2
    Ixy = Ix * Iy

    # Apply Gaussian blur for smoothing
    Sxx = cv2.GaussianBlur(Ixx, (3, 3), 0)
    Syy = cv2.GaussianBlur(Iyy, (3, 3), 0)
    Sxy = cv2.GaussianBlur(Ixy, (3, 3), 0)

    # Compute Harris response
    det = Sxx * Syy - Sxy**2
    trace = Sxx + Syy
    R = det - k * trace**2
```

```
    # Normalize for display
    R_norm = cv2.normalize(R, None, 0, 255,
                           cv2.NORM_MINMAX).astype(np.uint8)
    return R_norm
```

3.5 Characteristics of Different Image Regions

The Harris detector produces distinct responses for different image content:

For Flat Images: $R \approx 0$ (dark in R-map)

For Edge Images: $R < 0$ (near-black in R-map)

For Corner Images: $R > 0$ and large (bright/white in R-map)

The R-map thus provides a clear visual representation where corners appear as bright spots and edges/flat regions appear dark.

4 Part C: Hough Transform for Line Detection

4.1 Introduction to Hough Transform

The Hough transform is a technique for detecting parametric shapes (lines, circles, etc.) in images. For line detection, it transforms the image space to a parameter space where lines become points and their relationships become apparent.

4.2 Slope-Intercept Parameterization

A line in image space can be represented as:

$$y = mx + c$$

where m is the slope and c is the y-intercept. For each edge pixel (x_0, y_0), all lines passing through it satisfy:

$$y_0 = mx_0 + c$$

Rearranging to isolate c:

$$c = y_0 - mx_0$$

For a discrete set of slopes $m \in \{m_1, m_2, \ldots, m_N\}$, we compute the corresponding intercepts and accumulate votes in a 2D histogram (accumulator).

4.3 Algorithm Steps for Slope-Intercept Method

1. Apply Canny edge detection to obtain edge pixels
2. For each edge pixel (x, y):
   - For each slope m in a predefined range
   - Compute $c = y - mx$
   - Check if c is within valid range (from $-H$ to H where H is image height)
   - Increment accumulator at position (m, c)
3. Find peaks in the accumulator
4. Extract line parameters from peak positions

4.4 Accumulator Array

The accumulator is a 2D array:

$$A[m_{\text{idx}}, c_{\text{idx}}] = \text{number of edge pixels voting for line } (m, c)$$

Lines in the original image correspond to peaks in the accumulator space. The height of a peak indicates the number of collinear points.

4.4.1 Parameter Space Quantization

To create a discrete accumulator:

$$m_{\text{range}} = [m_{\min}, m_{\max}] \text{ discretized into } N_m \text{ bins}$$

$$c_{\text{range}} = [-H, H] \text{ discretized into } N_c \text{ bins}$$

For a 300×300 image with slope range $[-5, 5]$ and 100 slope bins:

$$\Delta m = \frac{10}{100} = 0.1, \quad \Delta c = \frac{2H}{N_c}$$

4.5 Slope-Intercept Implementation

```
def hough_slope_intercept(image, m_range=(-5, 5), m_bins=100):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 50, 150)

    h, w = edges.shape
    accumulator = np.zeros((m_bins, 2*h))

    # Get edge pixels
    y_coords, x_coords = np.where(edges > 0)

    # Quantize slopes
    slopes = np.linspace(m_range[0], m_range[1], m_bins)
```

```
    # Vote in accumulator
    for x, y in zip(x_coords, y_coords):
        for m_idx, m in enumerate(slopes):
            c = y - m * x
            c_idx = int(c + h)
            if 0 <= c_idx < 2*h:
                accumulator[m_idx, c_idx] += 1

    return accumulator, slopes
```

4.6 Rho-Theta Parameterization

An alternative parameterization uses polar coordinates:

$$\rho = x\cos(\theta) + y\sin(\theta)$$

where ρ is the perpendicular distance from origin and θ is the angle. This representation is more numerically stable and handles vertical lines naturally.

4.7 Rho-Theta Implementation

```
def hough_rho_theta(image, theta_bins=180, rho_bins=200):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 50, 150)

    h, w = edges.shape
    max_rho = int(np.sqrt(h**2 + w**2))

    accumulator = np.zeros((rho_bins, theta_bins))

    y_coords, x_coords = np.where(edges > 0)

    # Quantized angles
    thetas = np.linspace(0, np.pi, theta_bins)

    for x, y in zip(x_coords, y_coords):
        for theta_idx, theta in enumerate(thetas):
            rho = x * np.cos(theta) + y * np.sin(theta)
            rho_idx = int(rho * rho_bins / (2 * max_rho))
            if 0 <= rho_idx < rho_bins:
                accumulator[rho_idx, theta_idx] += 1

    return accumulator, thetas
```

4.8 Performance Comparison

The rho-theta method has several advantages over slope-intercept:

- **Numerical Stability**: Avoids infinity for vertical lines

- **Uniform Representation**: All lines treated uniformly
- **Computational Efficiency**: Slightly faster (one trigonometric function per pixel)
- **Better Clustering**: Lines cluster better in parameter space

The slope-intercept method is simpler conceptually but suffers from:

- Singularity at vertical lines ($m = \infty$)
- Non-uniform accumulator bins (spread increases with magnitude)
- Numerical instability for near-vertical lines

4.9 Line Extraction and Visualization

After computing the accumulator:

```
def extract_lines_from_accumulator(accumulator, slopes, threshold
    ):
    # Find peaks
    lines = []
    h_peaks, w_peaks = np.where(accumulator > threshold)

    for h_idx, w_idx in zip(h_peaks, w_peaks):
        m = slopes[h_idx]
        c = w_idx - accumulator.shape[1]//2
        lines.append((m, c))

    return lines

def draw_lines(image, lines):
    output = image.copy()
    h, w = image.shape[:2]

    for m, c in lines:
        # Compute line endpoints
        x0 = 0
        y0 = int(c)
        x1 = w
        y1 = int(m * w + c)

        cv2.line(output, (x0, y0), (x1, y1), (0, 255, 0), 2)

    return output
```

5 Conclusion

This assignment covers three fundamental techniques in computer vision:

- **Part A**: Gradient-based corner detection using Sobel operators and scatter analysis

- **Part B**: Harris corner response computation through eigenvalue analysis of the structure tensor
- **Part C**: Hough transform for parametric line detection in both slope-intercept and rho-theta spaces

These techniques are foundational for higher-level computer vision tasks including object detection, image registration, and structure from motion. The Harris detector provides precise corner locations while the Hough transform enables robust line detection even with incomplete or noisy edge information.

The implementation demonstrates the importance of parameter quantization, accumulation strategies, and threshold selection in feature detection pipelines.

Assignment: Machine Learning Algorithms from Scratch

Machine Learning Course

General Instructions

- Implement all algorithms from scratch using only `numpy`, `matplotlib`, and standard Python libraries.
- **Constraint:** No `scikit-learn` allowed for model implementation or scaling (except for K-Means clustering).
- Every step (scaling, loss, gradients, optimization) must be manually derived and implemented.

---

1 Topic 1: Linear Regression with L_2 Regularization

Linear regression models the relationship between a dependent variable y and features X. To prevent overfitting, we add a penalty term for large weights (L_2 Regularization / Ridge Regression).

1.1 Data Standardization

Before training, features must be standardized manually to have $\mu = 0$ and $\sigma = 1$:

$$z = \frac{x - \mu}{\sigma}$$

1.2 The Objective Function (Cost)

The Mean Squared Error (MSE) with Ridge Regularization is defined as:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^{n} \theta_j^2$$

where $h_\theta(x) = X\theta$.

1.3 Gradient Descent Update

To minimize $J(\theta)$, the weights are updated iteratively:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

The gradient for θ_j ($j > 0$) is:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j$$

2 Topic 2: Logistic Regression for Binary Classification

Logistic Regression predicts the probability $P(y = 1|x)$ using the sigmoid function.

2.1 The Sigmoid Function

Maps any real-valued number into the range $[0, 1]$:

$$g(z) = \frac{1}{1 + e^{-z}}$$

2.2 Binary Cross-Entropy Loss

The cost function with L_2 regularization is:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^{n} \theta_j^2$$

3 Topic 3: K-Means Clustering (Matrix Elements)

In this task, we cluster individual elements of a random matrix $M_{n \times m}$ based on their values.

3.1 Algorithm Steps

1. **Initialization:** Randomly pick k centroids.
2. **Assignment:** Assign each element $M_{i,j}$ to the nearest centroid:

$$c^{(i,j)} := \arg\min_k \|M_{i,j} - \mu_k\|^2$$

3. **Update:** Recompute centroids μ_k as the mean of all elements assigned to that cluster.

3.2 Expected Output

- **Assignment Table:** A matrix of shape (n, m) containing cluster IDs.
- **Cookbook:** A dictionary mapping `cluster_id` $\rightarrow$ list of (i, j) coordinates.
- **Centroids:** Final values of the cluster centers.

Custom PyTorch Dataset and DataLoader for Fruits-360

Machine Learning Assignment

1 Objective

The objective of this assignment is to design and implement a custom PyTorch Dataset and DataLoader for the Fruits-360 dataset. The pipeline includes lazy loading, fruit segmentation, handcrafted feature extraction, and batch processing for machine learning models.

2 Dataset Description

The Fruits-360 dataset consists of RGB images of fruits organized into class-wise folders. Each folder corresponds to a unique fruit category. The uniform white background allows reliable segmentation and feature extraction.

3 Design Principles

- Lazy loading of images
- No preloading into memory
- Manual feature extraction using OpenCV
- Modular and readable code

4 Complete Implementation

4.1 Dataset Download

```
import kagglehub

path = kagglehub.dataset_download("moltean/fruits")
print("Path to dataset files:", path)
```

4.2 Imports

```
import os
import cv2
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt
```

4.3 Custom Dataset Class

```
class FruitDataset(Dataset):
    def __init__(self, root_dir):
        self.root_dir = root_dir
        self.image_paths = []
        self.labels = []
        self.class_to_idx = {}

        classes = sorted(os.listdir(root_dir))

        for idx, class_name in enumerate(classes):
            class_path = os.path.join(root_dir, class_name)
            if not os.path.isdir(class_path):
                continue

            self.class_to_idx[class_name] = idx

            for img_name in os.listdir(class_path):
                img_path = os.path.join(class_path, img_name)
                self.image_paths.append(img_path)
                self.labels.append(idx)

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        img_path = self.image_paths[idx]
        label = self.labels[idx]

        image = cv2.imread(img_path)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        mask = get_fruit_mask(image)
        shape_features = get_shape_features(mask)
        color_features = get_color_features(image, mask)
        canny_image = get_canny_image(image)
        lbp_image = get_lbp_image(image)

        return (
            image,
```

```
            lbp_image,
            canny_image,
            mask,
            shape_features,
            color_features,
            label
        )
```

5 Utility Functions

5.1 Fruit Mask Generation

```
def get_fruit_mask(image):
    hsv = cv2.cvtColor(image, cv2.COLOR_RGB2HSV)

    lower_white = np.array([0, 0, 200])
    upper_white = np.array([180, 40, 255])

    white_mask = cv2.inRange(hsv, lower_white, upper_white)
    fruit_mask = cv2.bitwise_not(white_mask)

    return fruit_mask
```

5.2 Shape Feature Extraction

```
def get_shape_features(mask):
    contours, _ = cv2.findContours(
        mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )

    if len(contours) == 0:
        return [0, 0, 0, 0, 0, 0]

    cnt = max(contours, key=cv2.contourArea)

    area = cv2.contourArea(cnt)
    perimeter = cv2.arcLength(cnt, True)

    x, y, w, h = cv2.boundingRect(cnt)
    aspect_ratio = w / h if h != 0 else 0

    circularity = (4 * np.pi * area) / (perimeter ** 2) if
       perimeter != 0 else 0

    hull = cv2.convexHull(cnt)
    hull_area = cv2.contourArea(hull)
    solidity = area / hull_area if hull_area != 0 else 0

```

```
    rect_area = w * h
    extent = area / rect_area if rect_area != 0 else 0

    return [
        area,
        perimeter,
        aspect_ratio,
        circularity,
        solidity,
        extent
    ]
```

5.3 Color Feature Extraction

```
def get_color_features(image, mask):
    fruit_pixels = image[mask == 255]

    if fruit_pixels.size == 0:
        return [0, 0, 0, 0, 0, 0]

    R = fruit_pixels[:, 0]
    G = fruit_pixels[:, 1]
    B = fruit_pixels[:, 2]

    return [
        np.mean(R), np.mean(G), np.mean(B),
        np.std(R), np.std(G), np.std(B)
    ]
```

5.4 Canny Edge Detection

```
def get_canny_image(image):
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)
    edges = cv2.Canny(blurred, 100, 200)
    return edges
```

5.5 Local Binary Pattern

```
def get_lbp_image(image):
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
    h, w = gray.shape
    lbp = np.zeros((h, w), dtype=np.uint8)

    for i in range(1, h - 1):
        for j in range(1, w - 1):
            center = gray[i, j]
```

```
            binary = (
                (gray[i-1, j-1] >= center) << 7 |
                (gray[i-1, j]   >= center) << 6 |
                (gray[i-1, j+1] >= center) << 5 |
                (gray[i, j+1]   >= center) << 4 |
                (gray[i+1, j+1] >= center) << 3 |
                (gray[i+1, j]   >= center) << 2 |
                (gray[i+1, j-1] >= center) << 1 |
                (gray[i, j-1]   >= center)
            )
            lbp[i, j] = binary

    return lbp
```

6 DataLoader Integration

```
train_loader = DataLoader(
    dataset,
    batch_size=8,
    shuffle=True,
    num_workers=2
)
```

6.1 Batch Processing

```
for batch in train_loader:
    img, lbp, canny, mask, shape_f, color_f, lbl = batch

    shape_features_tensor = torch.stack(shape_f).T
    color_features_tensor = torch.stack(color_f).T

    print("img:", img.shape)
    print("lbp:", lbp.shape)
    print("canny:", canny.shape)
    print("mask:", mask.shape)
    print("shape:", shape_features_tensor.shape)
    print("color:", color_features_tensor.shape)
    print("label:", lbl.shape)

    break
```

7 Conclusion

This assignment demonstrates a complete data pipeline using PyTorch that combines classical computer vision techniques with modern deep learning workflows. The modular design ensures efficiency, readability, and extensibility.

1 Implementation Pipeline - Assignment 7

1.1 Data Preprocessing & Feature Engineering

To address the geospatial classification problem on Kepler-186f, we generated a synthetic dataset of 3,000 sensor readings. Since the safe zones exhibit radial symmetry (circular boundaries), we augmented the raw coordinate inputs (x, y) with a derived feature, radius $r = \sqrt{x^2 + y^2}$.

- **Input Features:** 3 dimensions (x, y, r).
- **Label Noise:** We injected 5% noise into the target labels to simulate sensor malfunction, forcing the model to generalize rather than memorize.
- **Data Split:** The data was partitioned into a Training Set (70%) and a Validation Set (30%).

1.2 Model Architecture (From Scratch)

Adhering to the constraint of avoiding `torch.nn`, we architected a Multi-Layer Perceptron (MLP) using raw PyTorch tensors.

- **Structure:** Input Layer (3 neurons) $\rightarrow$ 3 Hidden Layers (16 neurons each) $\rightarrow$ Output Layer (1 neuron).
- **Parameter Initialization:** Weights (W) and biases (b) were initialized randomly and registered as learnable parameters (`requires_grad=True`).

1.3 Forward Propagation

We implemented the forward pass via manual matrix operations. For any given layer l, the computation was defined as:

$$Z^{[l]} = A^{[l-1]} W^{[l]T} + b^{[l]} \tag{1}$$

$$A^{[l]} = g(Z^{[l]}) \tag{2}$$

We utilized **ReLU** activation for hidden layers to capture non-linearity and a **Sigmoid** activation at the final layer to output probability scores $\hat{y} \in [0, 1]$.

1.4 Loss Function Implementation

We manually implemented the **Binary Cross Entropy (BCE)** loss function. To ensure numerical stability and prevent 'NaN' values from log-zero calculations, we clamped predictions within a safe range $[\epsilon, 1 - \epsilon]$.

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right] \tag{3}$$

1.5 Optimization Strategy

With auto-optimizers forbidden, we implemented **Stochastic Gradient Descent (SGD)** manually. The training loop for each epoch proceeded as follows:

1. **Forward Pass:** Compute predictions and calculate loss $\mathcal{L}$.
2. **Backward Pass:** Trigger `loss.backward()` to compute gradients ($\frac{\partial \mathcal{L}}{\partial W}$) via Autograd.
3. **Parameter Update:** Within a `torch.no_grad()` block, update weights using the learning rate α:
$$W \leftarrow W - \alpha \cdot W.grad \tag{4}$$
4. **Gradient Reset:** Manually zero out gradients ($W.grad.zero\_()$) before the next iteration.